16-833: Robot Localization and Mapping, Spring 2026

Homework 3
Linear and Nonlinear SLAM Solvers

Po-Lin Chen
polinc@andrew.cmu.edu

1 2D Linear SLAM

1.1 Measurement Function

Odometry Measurement Function: Given robot poses $r_t = [r_{tx}, r_{ty}]^\top$ and $r_{t+1} = [r_{t+1,x}, r_{t+1,y}]^\top$ at time t and $t+1$, the odometry measurement function h_o represents the relative displacement between the two poses:

$$h_o(r_t, r_{t+1}) = r_{t+1} - r_t = \begin{bmatrix} r_{t+1,x} - r_{tx} \\ r_{t+1,y} - r_{ty} \end{bmatrix} \tag{1}$$

The Jacobian matrix H_o is the partial derivative of h_o with respect to the state vector $X = [r_{tx}, r_{ty}, r_{t+1,x}, r_{t+1,y}]^\top$:

$$H_o = \frac{\partial h_o}{\partial X} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{2}$$

Landmark Measurement Function: Given the robot pose $r_t = [r_{tx}, r_{ty}]^\top$ at time t and the k-th landmark $l_k = [l_{kx}, l_{ky}]^\top$, the landmark measurement function h_l represents the relative displacement from the robot to the landmark:

$$h_l(r_t, l_k) = l_k - r_t = \begin{bmatrix} l_{kx} - r_{tx} \\ l_{ky} - r_{ty} \end{bmatrix} \tag{3}$$

The Jacobian matrix H_l is the partial derivative of h_l with respect to the state vector $X = [r_{tx}, r_{ty}, l_{kx}, l_{ky}]^\top$:

$$H_l = \frac{\partial h_l}{\partial X} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{4}$$

1.2 Build a linear system

The function `create_linear_system` is implemented in `linear.py`.

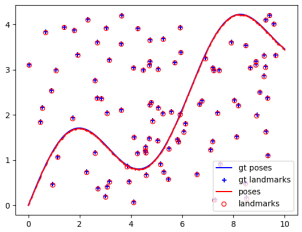
1.3 Solvers

The linear solvers are implemented in `solvers.py`.

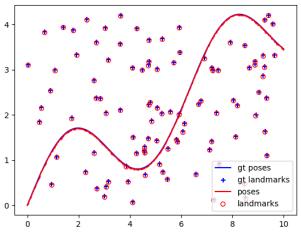
1.4 Exploit sparsity

All solvers (LU, QR, and their COLAMD versions) produced accurate trajectories and landmark positions that closely match the ground truth. This shows that the choice of solver affects speed, not accuracy, in a linear system.

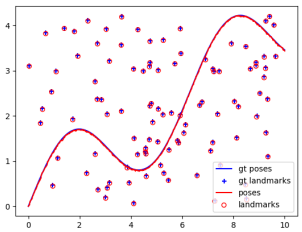
Results of processing `2d_linear.npz`:



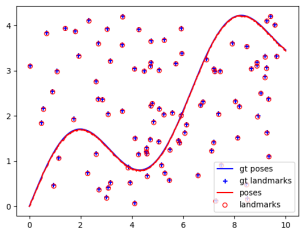
(a) QR



(b) QR COLAMD



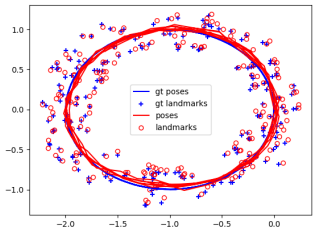
(c) LU



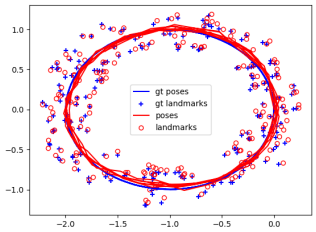
(d) LU COLAMD

- Efficiency Ranking: In terms of runtime, the general order of efficiency is LU > LU COLAMD > QR COLAMD > QR > Pseudo Inverse.
- Observations: (1) LU vs. QR: LU-class solvers are significantly faster than QR-class solvers because the LU algorithm is computationally less intensive for these linear systems. However, QR is often preferred in other contexts for its better numerical stability. (2) COLAMD Reordering: The COLAMD heuristic rearranges the matrix columns to increase sparsity. This sometimes reduces "fill-in" during decomposition, which usually speeds up the solving process. (3) Pseudo Inverse: This is the slowest method by far because it requires calculating $A^T A$ and its inverse, resulting in a high time complexity of $O(n^3)$.

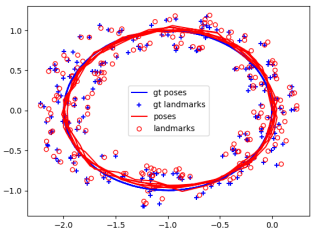
Results of processing `2d_linear_loop.npz`:



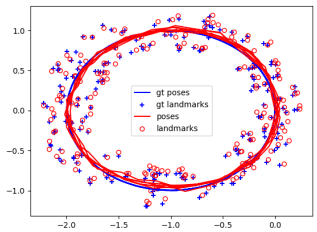
(a) QR



(b) QR COLAMD



(c) LU



(d) LU COLAMD

- Efficiency Ranking: The order of efficiency now is LU COLAMD > QR COLAMD > LU > QR.
- Observations: (1) The Loop Effect: In a loop dataset, the robot returns to a previous location and sees the same landmarks again. This creates many extra connections in the mathematical matrix, making it much more complex and denser than a simple linear path. (2) Why COLAMD is Critical: Because the loop matrix is dense, the solvers create a lot of "fill-in" (unnecessary non-zero numbers) during calculation. COLAMD reordering rearranges the matrix columns to keep the factors sparse (mostly zeros), which drastically reduces the computation time.

2 2D Nonlinear SLAM

2.1 Measurement Function

The Jacobian matrix H_l is derived by taking partial derivatives with respect to the state vector $X = [r_{tx}, r_{ty}, l_{kx}, l_{ky}]^\top$. Let $D = \sqrt{\Delta x^2 + \Delta y^2}$ be the range:

$$H_l = \frac{\partial h_l}{\partial X} = \begin{bmatrix} \frac{\partial \theta}{\partial r_{tx}} & \frac{\partial \theta}{\partial r_{ty}} & \frac{\partial \theta}{\partial l_{kx}} & \frac{\partial \theta}{\partial l_{ky}} \\ \frac{\partial d}{\partial r_{tx}} & \frac{\partial d}{\partial r_{ty}} & \frac{\partial d}{\partial l_{kx}} & \frac{\partial d}{\partial l_{ky}} \end{bmatrix} = \begin{bmatrix} \frac{\Delta y}{D^2} & -\frac{\Delta x}{D^2} & -\frac{\Delta y}{D^2} & \frac{\Delta x}{D^2} \\ -\frac{\Delta x}{D} & -\frac{\Delta y}{D} & \frac{\Delta x}{D} & \frac{\Delta y}{D} \end{bmatrix} \tag{5}$$

2.2 Build a Linear System

The function `create_linear_system` is implemented in `nonlinear.py`.

2.3 Solvers

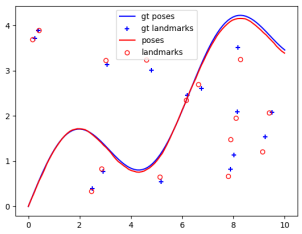
The key differences between the optimization processes are summarized below:

1. **Direct vs. Iterative**: In the linear case, the solution can be achieved in a single batch step using matrix decomposition. In the nonlinear case, there is

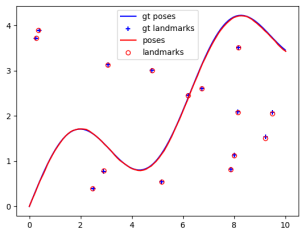
no closed-form solution, and the state must be updated incrementally through multiple iterations.

2. **Linearization**: Nonlinear SLAM requires continuous linearization of the measurement functions (Taylor series expansion) at the current estimate point before solving the system.

3. **Initialization**: Nonlinear optimization is highly sensitive to the initial estimate. While linear SLAM always finds the global minimum, nonlinear SLAM might get stuck in a local minimum if the initialization is poor.



(a) LU



(b) LU After Optimization